

Testes de Integração no Sistema de Agendamento de Voos de Helicóptero

Relatório técnico do case AeroLog Helicópteros: integração entre o serviço de agendamento, o módulo de validação e a persistência real em SQLite.

ALUNO	Luiz Gustavo Mella Massing
REPOSITÓRIO	<code>github.com/LuizGustavoMM/UniSenai/tree/main/qualidade-e-teste-de-software/teste-integracao</code>
AMBIENTE	Python 3.11.6 · pytest 8.3.3 · pytest-cov 5.0.0 · SQLite
SUÍTE	<code>tests/test_flight_integration.py</code> , 38 casos de teste
RESULTADO	38 testes aprovados · 100% de cobertura de instruções e de ramos em <code>src/</code>

1. Introdução e contextualização do case

A AeroLog Helicópteros opera táxi aéreo e transporte offshore para plataformas. O módulo auditado neste trabalho é o que agenda os voos: recebe a solicitação, decide se a operação é elegível e registra o voo na base. É um módulo pequeno em linhas de código e desproporcionalmente grande em consequência, porque uma falha ali não gera um erro de tela, gera um helicóptero decolando quando não deveria.

Os testes unitários não dão conta desse risco. Um teste unitário do módulo de validação pode provar que a regra de manutenção rejeita uma aeronave em inspeção, e mesmo assim o sistema agendar o voo, porque o serviço de agendamento pode simplesmente não consultar aquela regra, ou consultar e gravar assim mesmo. O defeito não mora dentro de nenhum componente, mora na costura entre eles.

Por isso a estratégia adotada foi de teste de integração com persistência real. Nada foi substituído por mock: o `BookingService` chama o `ValidationModule` de verdade, que consulta o `Database` de verdade, que escreve em um arquivo SQLite de verdade. O que cada teste verifica não é a chamada de um método, é o estado do banco depois da operação. A pergunta que a suíte responde é sempre a mesma: depois desta tentativa de agendamento, o que ficou gravado?

REGRAS DE NEGÓCIO IMPLEMENTADAS

As regras são avaliadas em ordem, e a primeira que reprova interrompe o processo: o fim do voo precisa ser posterior ao início; a aeronave precisa existir no cadastro; a manutenção precisa estar com status `LIBERADO`; o peso total não pode exceder a capacidade útil; e a aeronave não pode ter outro voo agendado no mesmo intervalo. Aprovado tudo, o voo é gravado com status `AGENDADO`.

2. Arquitetura da solução e integrações

O sistema tem três módulos internos conectados diretamente em Python, sem camada HTTP. A separação de responsabilidades é estrita: o validador lê o banco mas nunca escreve nele, e o serviço é o único que abre transação e grava.

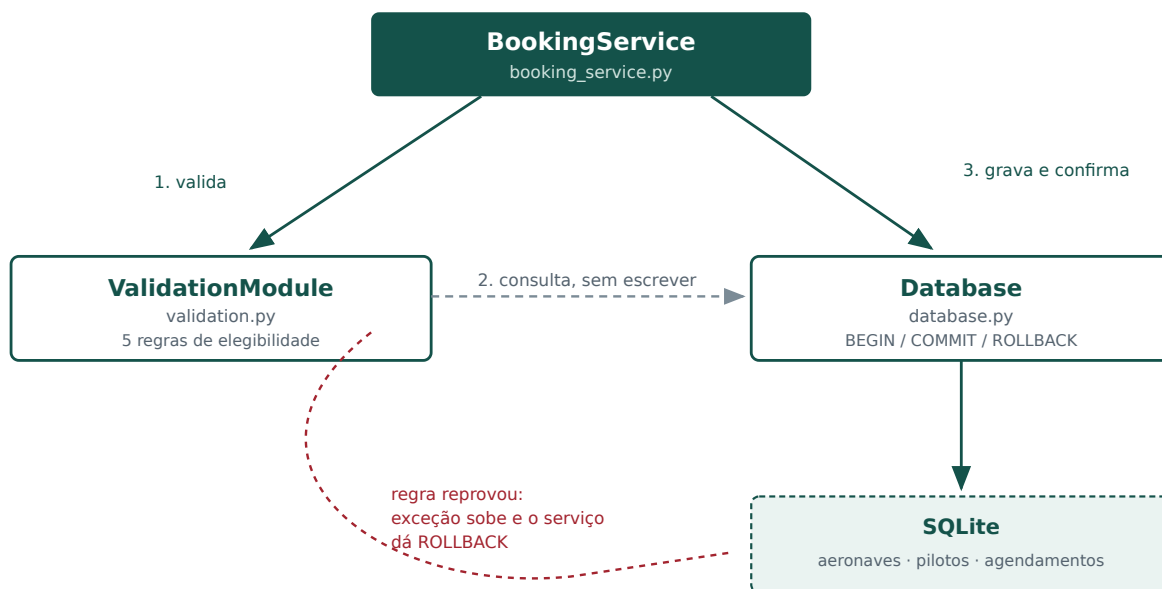


Figura 1. Fluxo de uma solicitação de agendamento. O serviço abre a transação antes de validar, de modo que qualquer reprovação desfaz tudo e o banco volta ao estado anterior.

Responsabilidades de cada módulo

MÓDULO	RESPONSABILIDADE E PONTOS DE INTEGRAÇÃO
database.py	Conexão SQLite e todas as operações sobre aeronaves, pilotos e agendamentos. Expõe <code>iniciar_transacao</code> , <code>confirmar</code> e <code>desfazer</code> , porque o controle de transação pertence a quem orquestra, não a quem persiste. A conexão usa <code>isolation_level=None</code> para que o <code>commit</code> seja sempre explícito.
validation.py	Aplica as cinco regras de elegibilidade consultando o Database. Levanta subclasses de <code>ErroDeValidacao</code> , o que permite ao chamador tratar qualquer reprovação de regra com um único <code>except</code> sem perder a distinção entre os motivos.
booking_service.py	Único ponto que escreve. Abre a transação, delega a validação e só grava se tudo for aprovado. Trata separadamente as reprovações de regra e as falhas inesperadas de escrita, mas em ambos os casos executa o <code>rollback</code> antes de repassar a exceção.

Detecção de conflito de horário

O conflito é resolvido no próprio SQL, e não em memória, para que a verificação enxergue tudo o que já está na base:

```
SELECT * FROM agendamentos
WHERE aeronave_id = ? AND status = 'AGENDADO'
AND inicio < ? AND fim > ?
```

Dois períodos se sobrepõem quando um começa antes do outro terminar e termina depois do outro começar. O uso de `<` e `>` em vez de `<=` e `>=` é deliberado: voos encostados, em que um termina exatamente no horário em que o outro começa, não são conflito. O caso CT05 existe para travar essa decisão.

3. Matriz de cenários de testes

A suíte tem 38 casos. Os quatro cenários obrigatórios do enunciado estão cobertos pelos grupos abaixo, acrescidos de casos complementares que fecham as regras restantes e os caminhos de erro. Todos os casos foram aprovados.

Cenário 1 · Caminho feliz, agendamento válido

CASO	DADOS DE ENTRADA	COMPORTAMENTO ESPERADO	OBTIDO
CT01	PT-HAA liberada, 08:00 às 10:00, 800 kg	Retorno com id gerado e status AGENDADO	Passou
CT02	Mesmo voo, consulta ao banco após a chamada	Registro localizado com início, fim e peso idênticos	Passou
CT03	Mesmo voo, verificação da chave estrangeira	aeronave_id aponta para a aeronave do prefixo pedido	Passou
CT04	Dois voos na PT-HAA, 08:00 às 10:00 e 14:00 às 16:00	Ambos aceitos, dois registros na base	Passou
CT05	Valor limite. Segundo voo das 10:00 às 12:00, encostado no primeiro	Aceito, pois encostar não é sobrepor	Passou

Cenário 2 · Conflito de horário da aeronave

CASO	DADOS DE ENTRADA	COMPORTAMENTO ESPERADO	OBTIDO
CT06	Dois voos na PT-HAA no intervalo exato 08:00 às 10:00	Segundo levanta ConflitoDeHorario	Passou
CT07	Mesmo cenário, contagem de registros após a exceção	A base permanece com um único agendamento	Passou
CT08	Sobreposição parcial, segundo voo das 09:00 às 11:00	Exceção e nenhuma inserção	Passou
CT09	Mesmo cenário, inspeção da mensagem	A exceção cita o id e o período do voo existente	Passou
CT10	PT-HCC no mesmo horário da PT-HAA	Aceito, o conflito é por aeronave e não por horário global	Passou

Cenário 3 · Aeronave em manutenção preventiva ou corretiva

CASO	DADOS DE ENTRADA	COMPORTAMENTO ESPERADO	OBTIDO
CT11	PT-HBB com status EM_INSPECAO	Levanta AeronaveEmManutencao	Passou
CT12	Mesmo cenário, contagem antes e depois	Base vazia antes e depois, nenhuma alteração permanece	Passou

CASO	DADOS DE ENTRADA	COMPORTAMENTO ESPERADO	OBTIDO
CT13	Mesmo cenário, inspeção da mensagem	A exceção cita o status que bloqueou o voo	Passou
CT14	PT-HBB liberada no banco após a reprovação	O mesmo voo passa a ser aceito e é gravado	Passou

Cenário 4 · Consistência do banco com fixtures e rollback

CASO	DADOS DE ENTRADA	COMPORTAMENTO ESPERADO	OBTIDO
CT15	Primeira execução da sequência, grava um voo	Começa com zero registros e termina com um	Passou
CT16	Segunda execução, logo após a anterior	Começa novamente do zero, sem herdar o registro	Passou
CT17	Terceira execução da mesma sequência	Mesmo comportamento, isolamento confirmado	Passou
CT18	Inspeção do caminho do arquivo de banco	Cada teste recebe um arquivo próprio no seu tmp_path	Passou
CT19	Inspeção da frota cadastrada pela fixture	As três aeronaves e os pilotos são recriados a cada teste	Passou
CT20	Voo confirmado seguido de tentativa que falha	O rollback da falha não apaga o registro já confirmado	Passou
CT21	Fecha a conexão e reabre o mesmo arquivo	O registro continua lá, a persistência é real em disco	Passou

Regras complementares de elegibilidade

CASO	DADOS DE ENTRADA	COMPORTAMENTO ESPERADO	OBTIDO
CT22	PT-HCC, capacidade 600 kg, carga de 601 kg	CapacidadeExcedida e base intacta	Passou
CT23	Valor limite. PT-HCC com exatamente 600 kg	Aceito, o limite é inclusivo	Passou
CT24	Prefixo PT-XXX, fora do cadastro	AeronaveNaoEncontrada e base intacta	Passou
CT25	Início 10:00 e fim 08:00	PeriodoInvalido e base intacta	Passou
CT26	Valor limite. Início igual ao fim	PeriodoInvalido, duração zero não é voo	Passou
CT27	Qualquer reprovação, capturada pela classe base	Todas derivam de ErroDeValidacao	Passou
CT28	Validador chamado isoladamente, com dados válidos	Aprova sem gravar nada, a escrita é do serviço	Passou

Consulta, cancelamento e comportamento do módulo de banco

CASO	DADOS DE ENTRADA	COMPORTAMENTO ESPERADO	OBTIDO
CT29	Dois voos inseridos fora de ordem cronológica	A agenda volta ordenada pelo horário de início	Passou
CT30	Consulta de prefixo inexistente	Lista vazia, sem exceção	Passou
CT31	Cancela um voo e reagenda o mesmo intervalo	Status vira CANCELADO e o horário é liberado	Passou
CT32	Cancelamento de id inexistente	Devolve False sem alterar nada	Passou
CT33	Operação no banco sem ter chamado conectar	RuntimeError com mensagem explicativa	Passou
CT34	Fechar a conexão duas vezes seguidas	Operação segura, sem exceção	Passou
CT35	Listagem sem filtro de aeronave	Traz os agendamentos de toda a operação	Passou
CT36	Atualização de status em prefixo inexistente	Zero linhas afetadas	Passou

Falhas inesperadas de escrita

CASO	DADOS DE ENTRADA	COMPORTAMENTO ESPERADO	OBTIDO
CT37	Agendamento com piloto_id 9999, que viola a chave estrangeira	IntegrityError repassado após rollback, base sem o registro	Passou
CT38	Falha simulada de escrita durante o cancelamento	Rollback e o voo permanece com status AGENDADO	Passou

4. Evidências de execução e cobertura

Os testes foram executados no Windows com Python 3.11.6 e pytest 8.3.3. A configuração fica no `pytest.ini`, com `pythonpath` apontando para a raiz do projeto para que os testes importem `src` sem instalação prévia do pacote.

```
pytest -v
pytest --cov=src --cov-report=term-missing
```

```
platform win32 -- Python 3.11.6, pytest-8.3.3, pluggy-1.6.0 -- C:\Users\Guu\Documents\GitPessoal\uniSenai\qualid.
cachedir: .pytest_cache
rootdir: C:\Users\Guu\Documents\GitPessoal\uniSenai\qualidade-e-teste-de-software\teste-integracao
configfile: pytest.ini
testpaths: tests
plugins: cov-5.0.0
collected 38 items

===== 38 passed in 1.79s =====
```

Figura 2. Abertura e fechamento da execução com `pytest -v`. 38 casos coletados e 38 aprovados em 1,79 s.

```
tests/test_flight_integration.py::TestCenario1CaminhoFeliz::test_ct01_agendamento_valido_retorna_dados_confirmados PASSED
tests/test_flight_integration.py::TestCenario1CaminhoFeliz::test_ct02_agendamento_valido_e_localizado_no_banco PASSED
tests/test_flight_integration.py::TestCenario1CaminhoFeliz::test_ct03_agendamento_valido_vincula_a_aeronave_correta PASSED
tests/test_flight_integration.py::TestCenario1CaminhoFeliz::test_ct04_dois_voos_em_horarios_distintos_sao_aceitos PASSED
tests/test_flight_integration.py::TestCenario1CaminhoFeliz::test_ct05_voo_encostado_no_anterior_nao_e_conflito PASSED
tests/test_flight_integration.py::TestCenario2ConflitoDeHorario::test_ct06_horario_identico_levanta_conflito PASSED
tests/test_flight_integration.py::TestCenario2ConflitoDeHorario::test_ct07_segundo_agendamento_nao_e_inserido_na_base PASSED
tests/test_flight_integration.py::TestCenario2ConflitoDeHorario::test_ct08_sobreposicao_parcial_tambem_e_conflito PASSED
tests/test_flight_integration.py::TestCenario2ConflitoDeHorario::test_ct09_mensagem_de_conflito_identifica_o_voo_existente PASSED
tests/test_flight_integration.py::TestCenario3AeronaveEmManutencao::test_ct10_aeronave_diferente_no_mesmo_horario_e_aceita PASSED
tests/test_flight_integration.py::TestCenario3AeronaveEmManutencao::test_ct11_aeronave_em_inspecao_levanta_excecao PASSED
tests/test_flight_integration.py::TestCenario3AeronaveEmManutencao::test_ct12_nenhuma_alteracao_permanece_no_banco PASSED
tests/test_flight_integration.py::TestCenario3AeronaveEmManutencao::test_ct13_mensagem_informa_o_status_que_bloqueou PASSED
tests/test_flight_integration.py::TestCenario3AeronaveEmManutencao::test_ct14_aeronave_liberada_apos_a_inspecao_passa_a_voar PASSED
```

Figura 3. Trecho da execução detalhada, com os casos dos Cenários 1, 2 e 3 nomeados individualmente. A nomenclatura `test_ct##` liga cada linha do terminal à matriz da seção anterior.

```
----- coverage: platform win32, python 3.11.6-final-0 -----
Name                               Stmts  Miss Branch BrPart  Cover   Missing
-----
src\booking_service.py             33      0      2      0  100%
src\database.py                    80      0      6      0  100%
src\validation.py                   36      0     10      0  100%
-----
TOTAL                             149      0     18      0  100%

1 empty file skipped.
===== 38 passed in 2.21s =====
```

Figura 4. Relatório do `pytest-cov` com cobertura de ramos habilitada. Os três módulos de `src/` atingem 100%, com `Miss = 0` e `BrPart = 0`.

MÓDULO	INSTRUÇÕES	NÃO COBERTAS	RAMOS	RAMOS PARCIAIS	COBERTURA
src/booking_service.py	33	0	2	0	100%
src/database.py	80	0	6	0	100%
src/validation.py	36	0	10	0	100%
Total	149	0	18	0	100%

A coluna `BrPart` zerada é o dado mais relevante da tabela. Ela indica que não há nenhuma condição avaliada em um só sentido: todo `if` do código foi exercitado tanto no caminho verdadeiro quanto no falso.

5. Conclusão e lições aprendidas

A suíte fechou com 38 casos aprovados e cobertura total de instruções e de ramos. Mais importante que o número, os quatro cenários obrigatórios ficaram com asserção sobre o estado do banco, e não apenas sobre o retorno do método. Em três dos quatro cenários a verificação principal é uma contagem de registros depois da operação.

O isolamento do banco foi o ponto mais difícil

A primeira tentação em um teste com banco é usar SQLite em memória, com `:memory:`. É rápido e o isolamento sai de graça, porque a base morre junto com a conexão. O problema é que esse atalho tira do teste justamente o que se quer testar: com banco em memória o teste nunca exercita a abertura de arquivo, e o comportamento de commit e rollback deixa de ser comparável ao de produção.

A solução foi usar arquivo em disco dentro do `tmp_path` do pytest, que gera um diretório temporário por teste e o remove ao final. O isolamento passa a vir do sistema de arquivos, não do ciclo de vida da conexão. O caso CT21 existe para provar que a escolha valeu a pena: ele fecha a conexão, reabre o mesmo arquivo com outra instância do `Database` e confirma que o registro continua lá. Esse teste seria impossível com banco em memória.

O controle de transação precisou sair do módulo de banco

A implementação inicial deixava cada operação de escrita com commit automático. Isso torna impossível garantir o requisito do Cenário 3, porque quando a regra reprova a gravação já aconteceu. A correção foi abrir a conexão com `isolation_level=None` e expor `iniciar_transacao`, `confirmar` e `desfazer` no `Database`, deixando o `BookingService` abrir a transação antes de validar.

Isso parece piorar o encapsulamento, já que o módulo de banco passa a expor detalhe de transação. Na prática é o contrário: quem sabe o que é uma operação atômica do ponto de vista do negócio é o serviço, não o repositório. Agendar um voo é uma operação só, mesmo que envolva consultas e uma escrita.

Cobrir o caminho de erro exigiu provocar a falha

Os dois últimos casos, CT37 e CT38, foram os que fecharam a cobertura em 100%. O bloco `except Exception` do serviço, que trata falhas que não são de regra de negócio, só é alcançado quando a escrita realmente quebra. Para o CT37 bastou usar um `piloto_id` inexistente, porque a chave estrangeira está ativa via `PRAGMA foreign_keys = ON`. Para o CT38 o método de banco foi substituído temporariamente por um que levanta exceção.

Esse par de testes mudou a leitura do resultado. Sem eles a suíte já passava e a cobertura de ramos já era de 100%, o que daria a impressão de trabalho concluído. O caminho que faltava não era um ramo condicional, era um bloco de tratamento de exceção, que a métrica de ramos não acusa. É um lembrete útil de que cobertura alta é condição necessária e não suficiente, e que vale olhar a coluna de instruções não cobertas mesmo quando a de ramos está zerada.

Código-fonte, suíte de testes e instruções de execução estão no repositório indicado na capa.